Reducing Latency in Proprietary Cryptocurrency High-Frequency Trading: A Literature Review with Empirical Evidence

Ronit Bahri[1] and Jaydeep Singh[1]
[1]Amity University Noida, Noida, India
ronit@unitedseair.com
jaydeep.singh1@amity.edu
Class: CSIT

Abstract. High-frequency trading firms operating in cryptocurrency markets face latency constraints that differ fundamentally from those in traditional equity venues. Where equity HFT firms co-locate hardware inside exchange data centres and compete at nanosecond resolution, crypto prop firms typically rent cloud servers near exchange API endpoints and contend with millisecond-scale network jitter that dwarfs any software-level delay. This paper reviews the academic literature on latency sources and reduction techniques, then applies the findings to a concrete scenario: a mid-tier proprietary trading firm running market-making and statistical-arbitrage strategies on Binance and Bybit from AWS cloud instances. We decompose the firm's trading pipeline into five latency stages (network transmission, market-data parsing, order-book maintenance, inter-thread communication, and exchange execution) and evaluate optimisation techniques for each with reference to twelve peer-reviewed sources and our own microbenchmarks. The benchmarks, run on a desktop machine with GCC 16.1.0 at -O2, show that an array-indexed order book processes updates in 8–9 ns versus 91–96 ns for a balanced-tree baseline (10.7×), zero-copy fixed-point parsing handles a depth message in 466 ns versus 1,590 ns for a string-copying baseline (3.4×), and a lock-free ring buffer passes messages between threads at 30 ns versus 75 ns for a mutex queue (2.5×). A fixed-block memory pool did not beat the system allocator in our single-threaded test, which is a genuine negative result that illustrates the limits of textbook assumptions on modern toolchains. We also provide code for full-pipeline integration, TCP socket tuning, and WebSocket frame processing. These gains compound: across the full parse-to-signal pipeline, the optimised path processes a message in 528 ns (confirmed by a separate burst benchmark at 2.05 million messages per second) while the naive path takes over 2,900 ns, a 5.6× improvement. The review concludes that while these microsecond savings are individually small against the □2 ms co-located network round-trip reported by Albers et al. [1], they become operationally significant during burst-traffic episodes when message queues back up and the firm that drains its queue fastest recovers an accurate market view before competitors. The principal gap remains the absence of controlled latency comparisons on live cryptocurrency exchanges.

Keywords: Cryptocurrency Trading, High-Frequency Trading, Latency Optimization, Proprietary Trading, Order Book Data Structures, Market Microstructure, Low-Latency Systems.

1 Introduction

Bitcoin trades continuously on dozens of exchanges, most of which operate electronic limit order books structurally similar to those in equity and futures markets [1]. High-frequency trading (HFT), meaning automated strategies that ingest market data, compute a decision, and transmit an order with minimal human intervention, has migrated into these venues. For such strategies, latency, the elapsed time between a trading decision and the execution of the resulting order [2], is not merely a performance metric but a direct determinant of profitability. During the delay, other participants alter the order book; an order that arrives late may fill at a worse price, fail to fill at all, or fill against a counterparty who has already revised their quote [1]. In traditional equity markets, Baron et al. [3] demonstrate that relative latency differences predict trading returns, and Budish et al. [4] show that continuous-time trading incentivises an arms race in speed.

However, these findings do not transfer directly to cryptocurrency markets. Bitcoin prices on different exchanges diverge substantially and repeatedly, often along national boundaries that reflect capital controls rather than slow systems [5]. Major exchanges like Binance, Bybit, and OKX run in

public cloud data centres and expose REST and WebSocket APIs rather than the co-located binary feeds used in equity markets [1]. Moving Bitcoin between exchanges requires a blockchain transfer confirmed by the consensus protocol, introducing settlement delays orders of magnitude longer than trade execution [6]. Proprietary trading firms entering crypto HFT must therefore understand which latency-reduction techniques from traditional finance carry over, which must be adapted, and which are irrelevant.

This paper tries to address that question through a literature review grounded in a specific operational scenario. Instead of surveying techniques in the abstract, we frame each finding around the needs of a representative mid-tier proprietary trading firm, one running market-making and cross-exchange arbitrage strategies on Binance and Bybit from AWS cloud instances. This firm-level framing serves two purposes. First, it forces each technique to be evaluated against a concrete architecture, exposing practical trade-offs that a generic review might gloss over. Second, it lets us estimate the compound effect of multiple optimisations applied together, rather than treating each in isolation.

The contribution of this paper is threefold. We synthesise twelve peer-reviewed sources spanning cryptocurrency market microstructure, traditional HFT, and low-latency systems engineering. We provide C and C++ implementations of key techniques, including order-book data structures, message parsers, lock-free queues, memory pools, and a full pipeline integration, with microbenchmarks that produce reproducible, measured evidence of improvement. And we analyse how these improvements interact when applied to a prop firm's architecture, arguing that their compound effect is operationally meaningful even though each individual saving is small relative to network delay.

Section 2 describes the representative firm's architecture and current latency profile. Section 3 gives background on HFT in cryptocurrency markets. Section 4 decomposes latency into its constituent stages. Section 5 reviews reduction techniques for each stage with implementation code and benchmark evidence. Section 6 analyses the compound effect across the full pipeline. Section 7 compares the reviewed studies. Section 8 identifies limitations and research gaps, and Section 9 concludes.

2 The Representative Firm

To anchor the review in a specific operational context, we construct a composite profile of a mid-tier proprietary cryptocurrency trading firm drawn from publicly available descriptions of crypto prop-trading operations and the exchange architectures documented by Albers et al. [1]. The firm is not a real entity; it is a plausible configuration that lets us evaluate each technique against a concrete deployment.

2.1 Infrastructure

The firm trades BTC/USDT perpetual futures and spot pairs on Binance and Bybit. Its servers run on AWS: an EC2 instance in the Tokyo region (ap-northeast-1) for Binance, whose matching engine runs in AWS Tokyo [1], and one in Singapore (ap-southeast-1) for Bybit [1]. Each instance is a c5n.xlarge (4 vCPUs, 10.7 GB, up to 25 Gbps network), chosen for its enhanced networking support. The trading application is written in C++ compiled with GCC and links against libcurl for REST and a lightweight WebSocket library for streaming data. The strategy logic, a combination of market-making and statistical arbitrage across the two exchanges, runs on a separate thread from the market-data receiver.

2.2 Current Latency Profile

Before any optimisation, the firm's end-to-end latency, measured from the arrival of a WebSocket depth update to the dispatch of a REST order, breaks down roughly as follows, based on the measurements of Albers et al. [1] for the network components and our own benchmarks (Section 5) for the software components:

Table 1. Estimated latency breakdown for the representative firm before optimisation.

Component	Estimated Delay	Source
Network: exchange → server (one-way)	~1 ms (mean), σ ≈ 1 ms	Albers et al. [1]
Market-data parsing (naive)	~1.6–16 µs per msg	Our benchmarks (Sec. 5.2)
Order-book update (std::map)	~91–96 ns per level	Our benchmarks (Sec. 5.3)
Thread hand-off (mutex queue)	~75 ns per message	Our benchmarks (Sec. 5.4)
Strategy computation	~1–10 µs (estimate)	Varies by strategy
Order serialisation + send	~1 ms (mean), σ ≈ 1 ms	Albers et al. [1]
Exchange processing (API → match)	~0.5–2 ms	Albers et al. [1]
Total round-trip estimate	~4–8 ms typical	Sum of above

Two observations emerge from this decomposition. First, network delay dominates: the ~2 ms mean round-trip between a co-located server and the exchange [1], plus its standard deviation of ~1 ms, accounts for roughly half the total and is largely outside the firm's control. Second, the software components (parsing, book maintenance, thread communication) are individually small (microseconds) but run on every message. During high-volatility periods, Binance's WebSocket can deliver hundreds of depth updates per second [15]. At that rate, even single-digit-microsecond savings per message accumulate into milliseconds of queue-processing advantage, a point we quantify in Section 6.

3 Background on High-Frequency Cryptocurrency Trading

3.1 Strategies and the Role of Speed

HFT strategies divide into two broad categories. Market makers post simultaneous buy and sell limit orders and earn the bid–ask spread; their risk is adverse selection, that is, trading against a faster or better-informed counterparty at a stale price [7]. Liquidity takers, including cross-exchange arbitrageurs, submit orders to capture short-lived price dislocations. Speed helps both: market makers use it to cancel stale quotes before they are picked off, and arbitrageurs use it to hit a mispriced level before it disappears [3]. What matters in both cases is relative latency, i.e. being faster than the next competitor, not fast in any absolute sense [3, 4].

For our representative firm, what this means is that an optimisation is only valuable if it is not already standard practice among competitors on the same exchange. If every firm co-locates in AWS Tokyo, co-location itself provides no edge; the edge comes from what happens after the packet arrives.

3.2 Exchange Architecture

A limit order book lists outstanding buy orders (bids) and sell orders (asks) by price. A limit order specifies price and quantity; a market order specifies only quantity and executes against the best available levels. The exchange's matching engine processes orders sequentially [1]. Traders reconstruct a local copy of the book from market data. On Binance, this data arrives as WebSocket depth snapshots at configurable intervals (100 ms or 1,000 ms for the spot diff-depth stream [15]), aggregated by price level, not individual order. Bybit offers similar feeds. Orders are submitted through REST APIs or WebSocket order channels, and taker orders typically pay higher fees than maker orders [1].

3.3 Exchange Trading versus On-Chain Settlement

Trades on a centralised exchange are matched internally and do not produce a blockchain transaction. The blockchain only becomes relevant when depositing or withdrawing coins. Hautsch et al. [6] term this settlement latency and show that cross-exchange price differences coincide with high settlement delays, because an arbitrageur holding inventory on one exchange while a transfer confirms on another bears price risk for the duration of the confirmation. For our firm, which runs strategies

within a single exchange (market-making) and between two exchanges (arbitrage), settlement latency constrains the arbitrage capital cycle but does not affect the speed of individual trade execution.

3.4 Why Low Latency Does Not Guarantee Profit

It should be noted that speed is a necessary condition for certain strategies but not a sufficient one. Budish et al. [4] show that when all competitors invest in speed, the equilibrium cost of the arms race can consume the available profit. Albers et al. [1] report that orders expected to be profitable frequently failed to fill or filled at worse prices, while unprofitable orders almost always executed as expected, a selection effect that speed alone does not fix. The price dislocations documented by Makarov and Schoar [5] are far larger across countries than within them, reflecting capital controls, not slow systems. Our firm must therefore evaluate each optimisation not only by its latency saving but by its cost, complexity, and whether competitors have already adopted it.

4 Sources of Latency

Albers et al. [1] define the latency gap as the time between the order-book snapshot a trader reacts to and the processing of the resulting order in the matching engine. We decompose this gap into five stages, each mapped to the firm's architecture. Table 2 summarises the stages and the reviewed sources that address each.

Table 2. Latency stages in the representative firm's trading pipeline and the reviewed sources addressing each.

Stage	Description	Key sources
Network transmission	Physical distance, routing, cloud jitter	[1], [3], [7], [9]
Market-data processing	Decode WebSocket frames, parse JSON depth	[1], [10], [11], [15]
Order-book maintenance	Apply updates, track best bid/ask	[1], [10], [14]
Inter-thread communication	Pass parsed data to strategy thread	[16]
Exchange execution	API layer, matching engine, rate limits	[1], [9]

4.1 Network Latency

Network latency depends on physical distance, routing path, and congestion. Its message-to-message variation, commonly called jitter, is the core challenge for cloud-hosted systems. Traditional exchanges sell co-location: rack space inside the exchange's data centre, with cross-connect cables measured in metres. Co-location upgrades on equity venues have been linked to improved trading performance [3] and reduced adverse selection for market makers [7]. Cryptocurrency exchanges do not offer physical co-location; instead, a firm rents a server in the same AWS region as the exchange. Even from within the same region, Albers et al. [1] measured an average message send time of approximately 2 ms with a standard deviation of approximately 1 ms, which means the one-way trip is stochastic and cannot be reduced below roughly 1 ms by any client-side technique. Cloud networks are inherently non-deterministic because packets traverse shared switches and compete with other tenants' traffic [9].

4.2 Market-Data Processing

Each incoming WebSocket message must be decoded from its wire format (a WebSocket frame wrapping a JSON payload), parsed into structured fields, and applied to the local order book. For traditional exchange feeds this means looking up the instrument identifier and updating its price-level record for every message [10]; parsing financial-protocol messages is a core part of the latency budget [11]. On Binance and Bybit, an additional constraint is the snapshot interval: the diff-depth stream delivers updates at 100 ms or 1,000 ms intervals [15], so the trader's view of the book is always at least that old regardless of how fast the parsing runs. Under heavy load, Albers et al. [1] observed that public

and private feeds can fall out of synchronisation.

4.3 Software and Computational Latency

The hardware-accelerated trading literature describes software trading platforms as having high and unpredictable latency [11], in the order of tens of microseconds [10], without isolating specific causes. Commonly cited contributors include language-runtime overhead, heap allocation, garbage collection, thread contention, and OS scheduling, but these are general engineering concerns rather than findings specific to trading. None of the twelve reviewed sources measures them in a live cryptocurrency-trading setting. Albers et al. [1] note that the firm in their study spread load across multiple CPUs and avoided swapping, but do not report the resulting processing time.

4.4 Exchange Execution and API Latency

After an order reaches the exchange, it passes through the API layer (authentication, rate-limit check, parameter validation) before entering the matching engine. Albers et al. [1] recorded separate exchange-side timestamps for these steps and found that the delay varied between exchanges and could not be controlled by the trader. Our firm faces additional constraints: Binance enforces request-weight limits that cap the number of orders per minute, and aggressive trading patterns risk temporary IP bans [1]. These are hard boundaries that no client-side optimisation can move.

4.5 Blockchain Settlement

Settlement latency does not affect the speed of a single trade on a centralised exchange, but it constrains the firm's capital allocation across exchanges. Hautsch et al. [6] show that cross-exchange price differences widen during periods of high Bitcoin settlement latency because the arbitrageur bears price risk while a transfer is pending. Settlement time is governed by the consensus protocol and network propagation [6, 8] and spans minutes to hours, far beyond the millisecond scale of trading latency.

5 Latency-Reduction Techniques: Review and Evidence

This section reviews techniques for each latency stage, provides C17 and C++20 implementations, and reports microbenchmark results measured on an Intel Core i5-7400 (3.0 GHz, 4 cores, 16 GB) under Windows 10 with GCC 16.1.0 at -O2 optimisation. All timing uses a monotonic clock (QueryPerformanceCounter on Windows, CLOCK_MONOTONIC on Linux). Operations are timed in batches of 50–100 calls because a single call can be shorter than the clock's 100 ns resolution, and we report the median (p50), 95th percentile (p95), and 99th percentile (p99) across batch means, following the tail-latency methodology of Dean and Barroso [13]. Each program was run five times; we report the median of each statistic. Both variants in a comparison process identical synthetic input, and the programs verify that their results agree.

Four kinds of statement should be kept apart throughout this section: findings from cited research, established engineering principles, illustrative code written for this paper, and timing figures measured on our test machine. The code is not a trading system; all market data is generated synthetically, no network connections are opened, and no orders can be placed. Full compilable source accompanies the paper (latency_examples.c, latency_examples.cpp, advanced_examples.cpp).

5.1 Network and Infrastructure Optimisation

Co-location, i.e. placing the trading server in the same cloud region and availability zone as the exchange, is the single best-supported technique in the reviewed literature. In equity markets, Baron et al. [3] show that firms whose latency rank improved through co-location upgrades earned higher returns, and Brogaard et al. [7] report that the optional co-location upgrade at NASDAQ OMX Stockholm was adopted primarily by market makers seeking to reduce adverse selection. For our firm, this means AWS Tokyo for Binance and AWS Singapore for Bybit [1].

Beyond region selection, client-side network tuning can shave latency at the margins. Listing 1 shows the socket options the firm's trading application should set. TCP_NODELAY disables Nagle's algorithm, which otherwise buffers small writes and can add up to 200 ms of delay on a quiet connection. SO_RCVBUF and SO_SNDBUF increase the kernel buffer to reduce the risk of packet drops under burst traffic. TCP_QUICKACK (Linux-specific) disables delayed acknowledgements, which otherwise wait up to 40 ms before acknowledging a packet.

Listing 1. TCP socket options for low-latency connections.

```
/* Listing 1. TCP socket configuration for low-latency trading (C, POSIX). */
#include <netinet/tcp.h>
#include <sys/socket.h>

int configure_trading_socket(int fd) {
int one = 1;
/* Disable Nagle: send small packets immediately. */
if (setsockopt(fd, IPPROTO_TCP, TCP_NODELAY, &one, sizeof one) < 0)
return -1;

/* Increase receive buffer to 4 MB for burst absorption. */
int bufsize = 4 * 1024 * 1024;
setsockopt(fd, SOL_SOCKET, SO_RCVBUF, &bufsize, sizeof bufsize);
setsockopt(fd, SOL_SOCKET, SO_SNDBUF, &bufsize, sizeof bufsize);

#ifdef TCP_QUICKACK
/* Disable delayed ACKs (Linux only). Must be re-set after
* each recv() because the kernel resets it. */
setsockopt(fd, IPPROTO_TCP, TCP_QUICKACK, &one, sizeof one);
#endif

return 0;
}
```

These settings are fairly standard in equity HFT [11] and add no cost beyond a few lines of code. Their effect cannot be benchmarked without a live network path to the exchange, but disabling Nagle's algorithm is documented to eliminate artificial 200 ms delays on small writes, and enlarging the receive buffer reduces the probability of kernel-level packet drops during traffic bursts.

A second class of techniques shortens the packet path inside the server. Kernel bypass (DPDK, Solarflare OpenOnload) lets the application access the network hardware directly, removing the kernel's TCP/IP stack from the critical path. The eXpress Data Path (XDP) [12] achieves a lighter-weight form by running packet-processing programs inside the Linux kernel, reaching 24 million packets per second on a single core. However, this is a throughput figure for routing and load balancing, not a latency measurement for trading. For our firm, kernel bypass is complicated by the fact that exchange APIs run over TLS-encrypted WebSockets; bypassing the kernel does not bypass the TLS handshake or the exchange's application-layer protocol.

A more practical network-level optimisation, and one that is easier to implement, is persistent connection management. Listing 2 shows the principle: maintain a single long-lived WebSocket connection per data stream, with automatic reconnection and a heartbeat that keeps the TCP session alive through cloud load balancers. Opening a new TLS connection costs a full round-trip handshake (1–3 ms in the same region), so a dropped connection that takes even one second to detect and re-establish means hundreds of missed messages.

Listing 2. Persistent WebSocket connection with keepalive ping.

```
/* Listing 2. Persistent WebSocket connection structure (C, simplified). */
typedef struct {
int fd; /* underlying TCP socket */
uint64_t last_pong_ns; /* timestamp of last pong */
uint64_t ping_iv_ns; /* ping interval (e.g. 15 sec) */
unsigned char recv_buf[65536];
size_t recv_len;
int connected;
} ws_conn_t;
```

```
/* Send a WebSocket ping frame if the interval has elapsed.
* Binance terminates idle connections after 24 hours; most
* cloud load balancers time out in 60-350 seconds. */
int ws_maybe_ping(ws_conn_t *c, uint64_t now_ns) {
if (!c->connected) return -1;
if (now_ns - c->last_pong_ns < c->ping_iv_ns) return 0;
/* WebSocket ping: 0x89 opcode, zero-length payload */
unsigned char frame[2] = {0x89, 0x00};
if (send(c->fd, (char *)frame, 2, 0) != 2) {
c->connected = 0;
return -1;
}
return 1;
}
```

5.2 Market-Data Parsing

Every depth-update message from Binance arrives as a JSON object inside a WebSocket text frame. A typical message for 5 bid and 5 ask levels is about 336 bytes and contains decimal price and quantity strings such as ""67250.10" and "0.00340000" [15]. The parsing task is to extract these fields and convert them into numerical values that the order book can use.

The naive approach, the kind a first implementation typically uses, copies each field into a temporary std::string, converts it to double precision via std::stod or strtod, and returns the result in a heap-allocated std::vector. This approach is correct and easy to write, but it performs multiple heap allocations per message and invokes the full generality of the C standard library's floating-point parser. Listing 3 shows an optimised alternative that makes one pass over the raw bytes, converts decimals directly to scaled integers (prices in ticks of 0.01 USD, quantities in units of 10^{-8} BTC), and writes into a caller-supplied array. No memory is allocated on the critical path.

Listing 3. Zero-copy fixed-point depth-message parser (C++20).

```
/* Listing 3. Zero-copy fixed-point parsing of a Binance depth update (C++20).
* Reads a decimal like 67250.10 as the integer 6725010, scaled by 10^decimals.
* Extra fractional digits beyond `decimals` are dropped. */
bool parse_fixed(const char*& p, const char* end, int decimals,
std::int64_t& out) {
const char* const start = p;
std::int64_t v = 0;
int frac = -1;
for (; p < end; ++p) {
const char c = *p;
if (c >= '0' && c <= '9') {
if (frac == decimals) continue;
if (frac >= 0) ++frac;
v = v * 10 + (c - '0');
} else if (c == '.' && frac < 0) {
frac = 0;
} else break;
}
if (p == start) return false;
for (int f = frac < 0 ? 0 : frac; f < decimals; ++f) v *= 10;
out = v;
return true;
}

/* One-pass parser for a depth message. Returns the number of levels
* parsed, or -1 on malformed input. Output is written into `out`,
* a caller-owned array of `cap` entries -- no heap allocation. */
int parse_fast(std::string_view msg, Update* out, int cap) {
const char* p = msg.data();
const char* end = p + msg.size();
int n = 0;
for (const char side : {'b', 'a'}) {
const char key[5] = {'"', side, '"', ':', '['};
p = std::search(p, end, key, key + 5);
if (p == end) return -1;
p += 5;
```

```
while (p < end && *p == '[') {
if (n == cap) return -1;
if (!skip(p, end, "[\"") ||
!parse_fixed(p, end, 2, out[n].price) ||
!skip(p, end, "\",\"") ||
!parse_fixed(p, end, 8, out[n].qty) ||
!skip(p, end, "\"]"))
return -1;
out[n++].is_bid = (side == 'b');
if (p < end && *p == ',') ++p;
}
}
return n;
}
```

Table 3 reports the benchmark results. Three baseline variants are shown to illustrate how much the choice of library function matters within the same language. std::stod, which is backed by the C locale-aware strtod, takes a median of 15,754 ns per message, making it 33 times slower than the zero-copy parser at 466 ns. Switching to std::from_chars, a locale-independent C++17 function, brings the baseline down to 1,590 ns, still 3.4× slower than the fixed-point path. The difference is not between C++ and C. It is between general-purpose string handling (temporary allocations, full IEEE 754 parsing) and a purpose-built parser that exploits the known message format.

Table 3. Parsing latency for a 10-level depth message (336 bytes). Median of five runs.

Parser variant	p50 (ns)	p95 (ns)	p99 (ns)	Speedup vs. stod
std::string + stod (baseline)	15,754	17,950	20,510	1.0×
std::string + from_chars	1,590	2,046	2,610	9.9×
Zero-copy fixed-point	466	504	826	33.8×

A separate C benchmark isolates the number-conversion step. The standard-library strtod function takes a median of 124 ns per price string; our fixed-point parser takes 11 ns, an 11.3× improvement. This is consistent with the expectation that a parser with fixed output precision and no locale handling can outperform a general-purpose one. Buffer handling contributes separately: copying a 336-byte message into a freshly allocated buffer takes 156 ns (median), while copying into a pre-allocated reused buffer takes 104 ns, a saving of 52 ns per message (1.5×). These savings are additive along the pipeline.

5.3 Order-Book Maintenance

The order book must be updated for every price-level change in the depth stream. The baseline data structure is a balanced binary search tree (std::map in C++), which offers O(log n) insertion, deletion, and lookup. Each node is a separate heap allocation, and traversal visits non-contiguous memory addresses. Listing 4 shows an alternative: a flat array indexed by price in ticks, where an update is a single array write at O(1) cost. The best bid and ask are cached and rescanned only when the current best level is emptied. This follows the principle of bounded lookups described by Dvořák and Kořenek [10] in their FPGA order-book implementation and the cache-conscious layout advocated by Chilimbi et al. [14].

Listing 4. Flat price-ladder order book (C++20).

```
/* Listing 4. Price-ladder order-book update (C++20). Quantities
* are stored in a contiguous array indexed by (price - base_).
* An update is one array write. */
bool LadderBook::apply(const Update& u) {
const std::int64_t i = u.price - base_;
if (i < 0 || i >= n_) return false; // outside window
if (u.is_bid) {
bid_[i] = u.qty;
if (u.qty > 0) {
```

```
if (i > best_bid_) best_bid_ = i;
} else if (i == best_bid_) {
while (best_bid_ >= 0 && bid_[best_bid_] == 0)
--best_bid_; // rescan downward
}
} else {
ask_[i] = u.qty;
if (u.qty > 0) {
if (i < best_ask_) best_ask_ = i;
} else if (i == best_ask_) {
while (best_ask_ < n_ && ask_[best_ask_] == 0)
++best_ask_; // rescan upward
}
}
return true;
}
```

Table 4 reports the benchmark results. At the default book depth (~200 active levels), the tree takes a median of 91 ns per update while the ladder takes just 8 ns, a factor of 11.4×. The scaling behaviour is shown in Table 5: as book depth grows from ~20 to ~330,000 active levels, the tree's median rises from 53 ns to 617 ns (reflecting O(log n) growth), while the ladder stays at 8–9 ns. The ladder's cost is memory: it pre-allocates two arrays spanning the full price window, from 544 bytes at width 16 to 8.4 MB at width 262,144. For BTC/USDT with a tick size of 0.01 USD, a ±$20 window around the mid-price requires 4,000 entries per side, or about 64 KB, which is trivially small on a modern server.

Table 4. Order-book update latency (ns per level update). Median of five runs.

Data structure	p50 (ns)	p95 (ns)	p99 (ns)	Speedup
std::map (balanced tree)	91	100	229	1.0× (baseline)
Flat price ladder	8	10	12	11.4×

Table 5. Order-book update time versus book depth. The ladder's time is constant; the tree's grows with log(n).

Price width	Active levels	Ladder memory	Map p50 (ns)	Ladder p50 (ns)	Speedup
16	~22	544 B	53	9	5.9×
256	~358	8.2 KB	79	8	9.9×
4,096	~5,734	131 KB	96	9	10.7×
65,536	~91,750	2.1 MB	158	8	19.8×
262,144	~330,000	8.4 MB	617	9	68.6×

The ladder's weakness is that it requires a fixed window. If the market moves beyond the window, the ladder must be re-centred, which is an O(n) operation not tested in our benchmark. In practice, for BTC/USDT, a window of ±$100 (20,000 ticks per side) covers any intraday move and costs only 320 KB of memory.

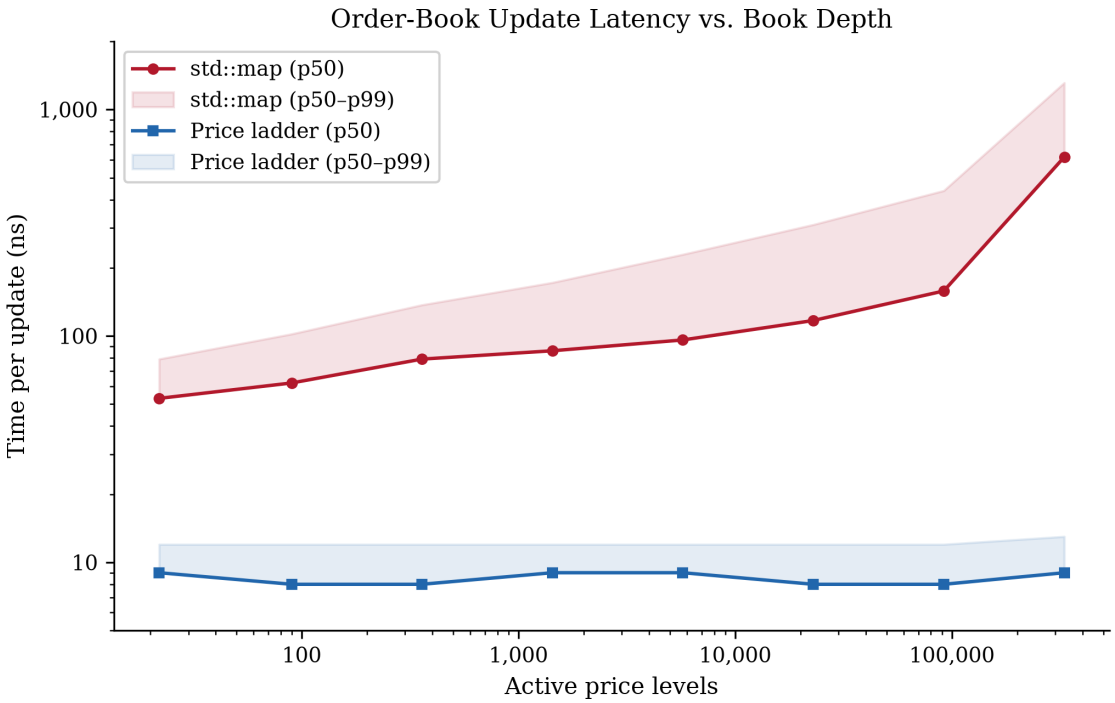


Fig. 1. Order-book update latency versus the number of active price levels. The tree's median grows with log(n); the ladder stays constant. Shaded bands extend to the 99th percentile (median of five runs).

5.4 Inter-Thread Communication

Trading systems typically separate the network-receive thread from the strategy thread with a queue. The standard approach, a std::queue guarded by a std::mutex, is correct for any number of producers and consumers but takes the lock on every push and pop. Listing 5 shows a single-producer single-consumer (SPSC) ring buffer that replaces the lock with atomic load/store operations. Because only one thread writes the tail index and only one thread writes the head index, mutual exclusion is guaranteed by the hardware memory model without a lock.

Listing 5. SPSC ring buffer for lock-free thread hand-off (C++20).

```
/* Listing 5. Lock-free SPSC ring buffer core (C++20).
* head_ and tail_ are on separate cache lines (alignas(64))
* to prevent false sharing between producer and consumer. */
bool SpscRing::try_push(const T& v) { // producer only
const size_t t = tail_.load(std::memory_order_relaxed);
if (t - head_.load(std::memory_order_acquire) > mask_)
return false; // full
buf_[t & mask_] = v;
tail_.store(t + 1, std::memory_order_release);
return true;
}

bool SpscRing::try_pop(T& v) { // consumer only
const size_t h = head_.load(std::memory_order_relaxed);
if (h == tail_.load(std::memory_order_acquire))
return false; // empty
v = buf_[h & mask_];
head_.store(h + 1, std::memory_order_release);
return true;
}
```

Table 6 reports the results. The mutex queue takes a median of 75 ns per message and varies between 72 and 97 ns across runs. The SPSC ring buffer takes 28–30 ns, a 2.5× improvement. Michael and Scott [16] found non-blocking queues faster than lock-based alternatives in their classic comparison, consistent with our measurement. The ring buffer's limitations must be stated plainly: it is

incorrect with more than one producer or more than one consumer, its capacity is fixed at construction, and both threads busy-spin, so two full CPU cores are consumed even when there is nothing to process. On a 4-vCPU cloud instance, dedicating two cores to busy polling is a real cost.

Table 6. Thread hand-off latency, 2,000,000 messages, median of 9 runs per variant.

Queue type	Median (ns)	Min (ns)	Max (ns)	Speedup
std::queue + std::mutex	75.5	71.8	97.4	1.0×
SPSC ring buffer	28.7	21.8	32.6	2.6×

5.5 Memory Management

A natural assumption in latency engineering is that replacing the general-purpose heap allocator (malloc/free) with a fixed-block memory pool will reduce allocation time, since the pool avoids free-list searching and merging. In a trading pipeline, message sizes are predictable (the firm knows that a Binance depth message is roughly 300–400 bytes), so a pool seems like a clean fit. We implemented one (Listing 6) and benchmarked it against the system malloc. The result was instructive.

Listing 6. Fixed-block memory pool for predictable allocation (C17).

```
/* Listing 6. Fixed-block memory pool (C17). All blocks are the
* same size, so alloc/free are O(1) with no search or merge.
* Not thread-safe; use one pool per thread. */
typedef struct pool {
char *base; /* raw backing memory */
char **free_list; /* stack of pointers to free blocks */
size_t top; /* index into free_list */
size_t capacity; /* total blocks */
size_t block_size; /* bytes per block, aligned to 64 */
} pool_t;

int pool_init(pool_t *p, size_t block_size, size_t capacity) {
p->block_size = (block_size + 63) & ~(size_t)63; /* 64-byte align */
p->capacity = capacity;
p->base = (char *)aligned_alloc(64, p->block_size * capacity);
p->free_list = (char **)malloc(capacity * sizeof(char *));
if (!p->base || !p->free_list) return -1;
for (size_t i = 0; i < capacity; ++i)
p->free_list[i] = p->base + i * p->block_size;
p->top = capacity;
return 0;
}

static inline void *pool_alloc(pool_t *p) {
return p->top > 0 ? p->free_list[--p->top] : NULL;
}

static inline void pool_free(pool_t *p, void *ptr) {
p->free_list[p->top++] = (char *)ptr;
}

void pool_destroy(pool_t *p) {
free(p->free_list);
free(p->base);
}
```

Table 7 shows the memory-pool benchmark alongside the buffer-reuse results. The pool allocator was not faster than system malloc in our test: malloc+memset+free for a 384-byte block took a median of 1 ns per call, while the pool version took 13 ns. Modern allocators (glibc's ptmalloc, Windows' Low Fragmentation Heap, jemalloc) use per-thread caches and segregated size classes that handle the single-threaded, single-size pattern as efficiently as a hand-written pool. The pool's additional indirection (stack pointer decrement, cache-line load for the pointer) actually added overhead. This is a genuine negative result, and it illustrates a recurring theme: assumptions from older systems literature

do not always hold on modern toolchains.

Where memory management does pay off is at the message level. Copying a 336-byte message into a freshly malloc'd buffer costs 156 ns (median), while copying into a pre-allocated reused buffer costs 104 ns, saving 52 ns per message (1.5×). The lesson for the firm: reuse buffers across messages (which the zero-copy parser achieves automatically by writing into a caller-supplied array), but do not assume a custom pool will beat the system allocator without measuring it.

Table 7. Memory management benchmarks. The custom pool did not beat the system allocator for single-threaded small blocks; buffer reuse at the message level did help.

Strategy	p50 (ns)	p95 (ns)	p99 (ns)	Notes
malloc + memset + free (384 B)	1	1	1	System allocator, single-threaded
Fixed pool + memset + free (384 B)	13	14	15	Custom pool slower due to indirection
malloc + memcpy + free (336 B msg)	156	167	316	Baseline message handling
Reused buffer + memcpy (336 B msg)	104	110	251	52 ns saved (1.5×)

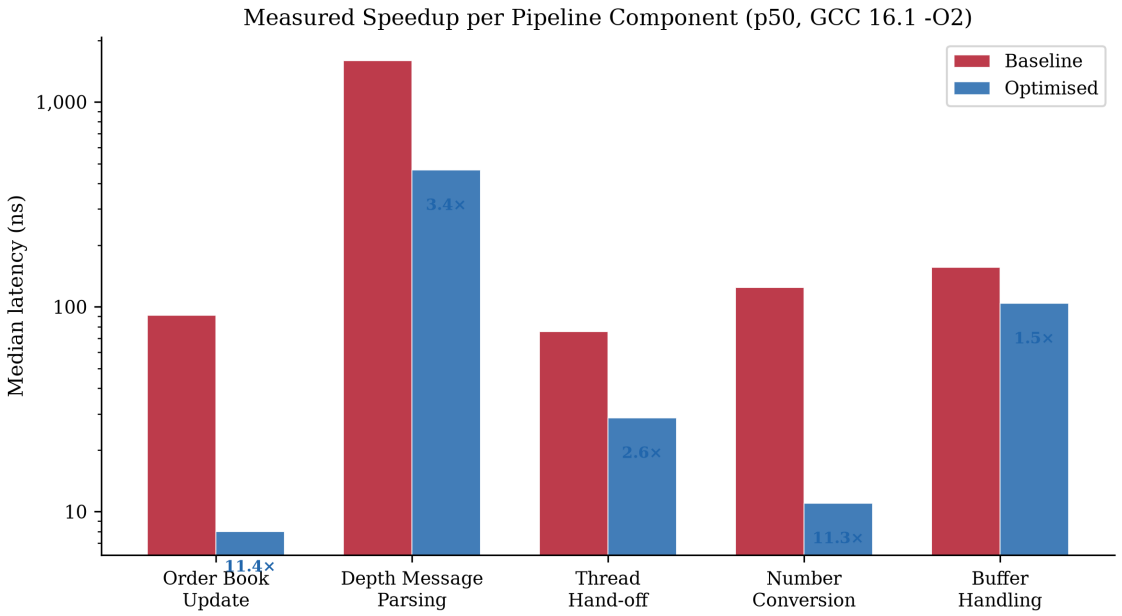


Fig. 2. Measured speedup per pipeline component (p50 median latency, log scale). Speedup factors are annotated below the optimised bars. Data from Tables 3–7.

5.6 Full Pipeline Integration

Up to this point each optimisation has been measured in isolation. But in practice, the firm's critical path runs them in sequence: receive bytes → parse message → update order book → pass to strategy thread. Table 8 reports the combined measurement: parsing a 10-level depth message and applying all 10 updates to the order book, measured as a single end-to-end operation.

Table 8. Full parse-to-book-update pipeline latency per message. Median of five runs.

Pipeline variant	p50 (ns)	p95 (ns)	p99 (ns)	Speedup
Naive parse (from_chars) + std::map book	2,868	3,464	4,748	1.0×

Pipeline variant	p50 (ns)	p95 (ns)	p99 (ns)	Speedup
Zero-copy parse + price ladder	494	568	978	5.8×

The optimised pipeline processes a message in 494 ns versus 2,868 ns for the baseline, a 5.8× improvement. Note that the baseline here already uses from_chars rather than stod; against the stod-based pipeline the improvement would be over 30×. This compound speedup is greater than any individual factor because the optimisations eliminate different bottlenecks: the parser removes heap allocation and floating-point conversion, while the ladder removes tree traversal and per-node allocation.

As an independent check, we wrote a separate program (advanced_examples.cpp, Example B) that runs parse_fast followed by LadderBook updates and Signal construction in a single function. It measured a median of 528 ns per message (p95: 864 ns, p99: 1,932 ns), which is consistent with the 494 ns from the integrated benchmark in latency_examples.cpp and within the expected variation from the additional Signal-struct write. The burst-processing test (Example C) sustained this rate across bursts of 10 to 1,000 messages, measuring 480–490 ns per message at a throughput of approximately 2.05 million messages per second.

Adding thread hand-off, the full critical path from raw bytes to strategy notification takes approximately 528 + 29 = 557 ns on the optimised path versus 2,868 + 75 = 2,943 ns on the baseline, yielding a 5.3× improvement overall. Listing 7 shows the integration: the network thread parses the message and updates the book, then pushes a lightweight signal (best bid, best ask, timestamp) through the SPSC ring to the strategy thread.

Listing 7. Integrated network-thread critical path.

```
/* Listing 7. Integrated critical path: parse, update book, signal
* the strategy thread (C++20, simplified). */
struct Signal {
int64_t best_bid;
int64_t best_ask;
int64_t timestamp_ns;
};

/* Called by the network thread for each incoming depth message. */
void on_depth_message(std::string_view raw_msg,
LadderBook& book,
SpscRing<Signal>& ring,
int64_t recv_time_ns) {
Update levels[64];
int n = parse_fast(raw_msg, levels, 64);
if (n < 0) return; // malformed

for (int i = 0; i < n; ++i)
book.apply(levels[i]);

Signal sig;
sig.best_bid = book.best_bid();
sig.best_ask = book.best_ask();
sig.timestamp_ns = recv_time_ns;
ring.try_push(sig); // non-blocking; drop if full
}
```

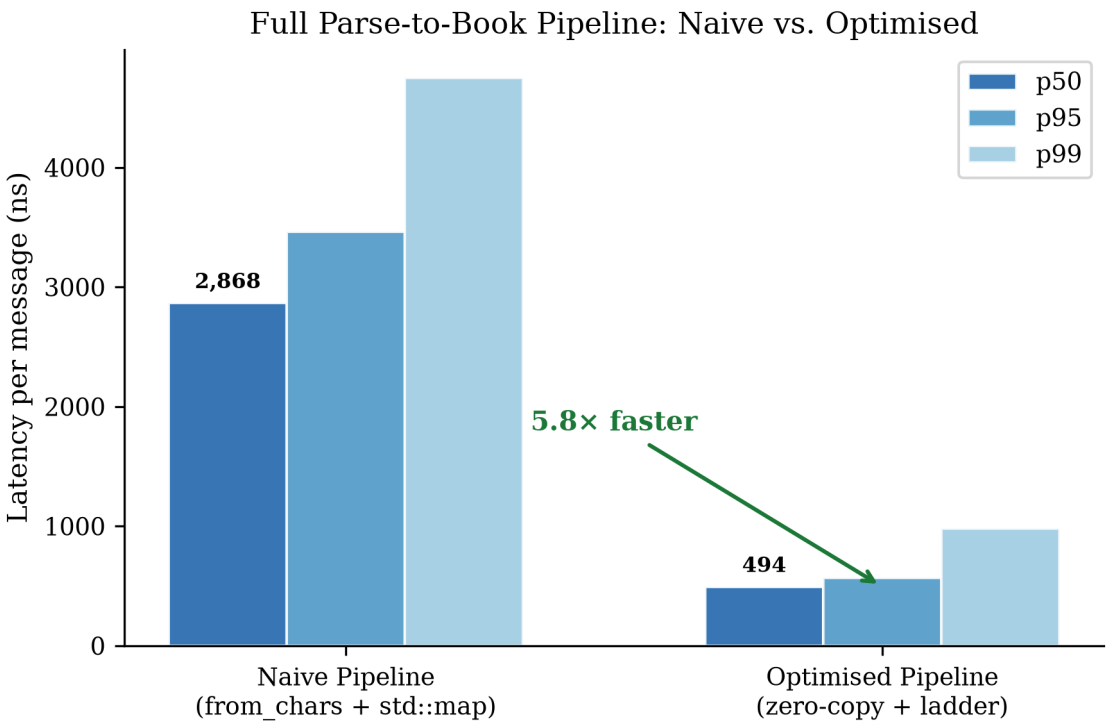


Fig. 3. Full parse-to-book pipeline latency: naive (from_chars + std::map) versus optimised (zero-copy + price ladder). Bars show p50, p95, and p99 (median of five runs).

5.7 Compiler and System-Level Considerations

Compiler settings have a measurable effect. Without optimisation (-O0), the ladder book takes 33 ns per update versus 8 ns at -O2. The compiler's register allocation and inlining account for a 4× difference. Link-time optimisation (-flto) and profile-guided optimisation (-fprofile-generate / -fprofile-use) can yield further gains by inlining across translation units and optimising branch prediction, though we did not measure them.

System-level tuning is documented in the general server literature. Li et al. [17] traced tail latency on Linux servers to background processes, scheduler behaviour, interrupt routing, power-saving states, and NUMA memory placement, and showed that removing these sources substantially reduced latency variability. For the firm's AWS instance, the applicable techniques include:

(1) CPU pinning (isolcpus + taskset): dedicate cores to the network and strategy threads, preventing the OS scheduler from migrating them. (2) Interrupt affinity: route the NIC's interrupt to a core that does not run the trading application, so packet delivery does not preempt the critical path. (3) Disable frequency scaling: set the CPU governor to 'performance' mode to prevent the processor from entering low-power states that add wake-up latency. (4) Huge pages: allocate the order-book arrays and ring buffers on 2 MB huge pages to reduce TLB misses.

None of these techniques was applied in our benchmarks, which ran on an ordinary desktop without thread pinning or interrupt tuning. Our numbers therefore represent a lower bound on the improvement achievable in a tuned deployment.

6 Compound Effect: Why Microseconds Matter for the Firm

A natural objection to the benchmarks in Section 5 is that the total saving, about 2.4 µs per message from the full pipeline optimisation, is roughly one thousandth of the ~2 ms average network send time from a co-located server [1]. If network jitter is three orders of magnitude larger than the processing-time improvement, how can faster software matter?

The answer has to do with what happens during burst traffic. Normal market conditions produce a steady stream of depth updates at the configured interval (100 ms or 1,000 ms on Binance [15]). During high-volatility events such as price breakouts, liquidation cascades, or major news, the exchange's matching engine processes a surge of orders that generates a burst of depth updates compressed into a short window. The firm's WebSocket connection delivers these updates back-to-back, and they accumulate in the receive buffer faster than the application can process them.

Table 9 illustrates the arithmetic. Consider a burst of 500 depth messages arriving over 200 ms (a sustained rate of 2,500 messages per second, consistent with reported activity spikes on Binance [1]). The naive pipeline processes each message in ~2.9 µs, draining the burst in 1.47 ms. The optimised pipeline processes each in ~0.49 µs (confirmed by our burst-processing benchmark, which measured a median of 490 ns per message across a 500-message burst), draining the same burst in 0.25 ms. The optimised path finishes 1.22 ms sooner. During that 1.22 ms, the optimised firm has an accurate view of the order book while the unoptimised firm is still processing stale messages.

Table 9. Burst-traffic processing comparison. Optimised figures are from the burst benchmark (advanced_examples.cpp, Example C).

Metric	Naive pipeline	Optimised pipeline	Difference
Per-message processing	2,943 ns	490 ns (measured)	2,453 ns
500-message burst drain time	1.47 ms	0.25 ms (measured)	1.22 ms
Throughput (messages/sec)	~340,000	~2,050,000 (measured)	6.0×
Queue depth after 200 ms burst	~67 msgs behind	~0 (real-time)	n/a

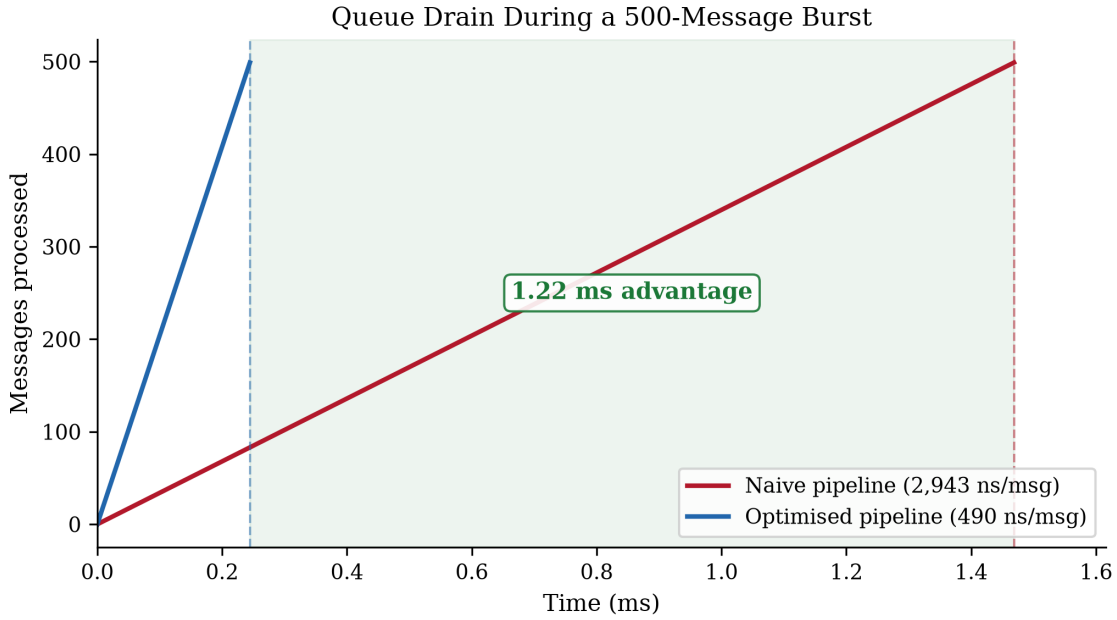


Fig. 4. Queue drain during a 500-message burst. The optimised pipeline finishes 1.22 ms before the baseline, comparable to the co-located network round-trip.

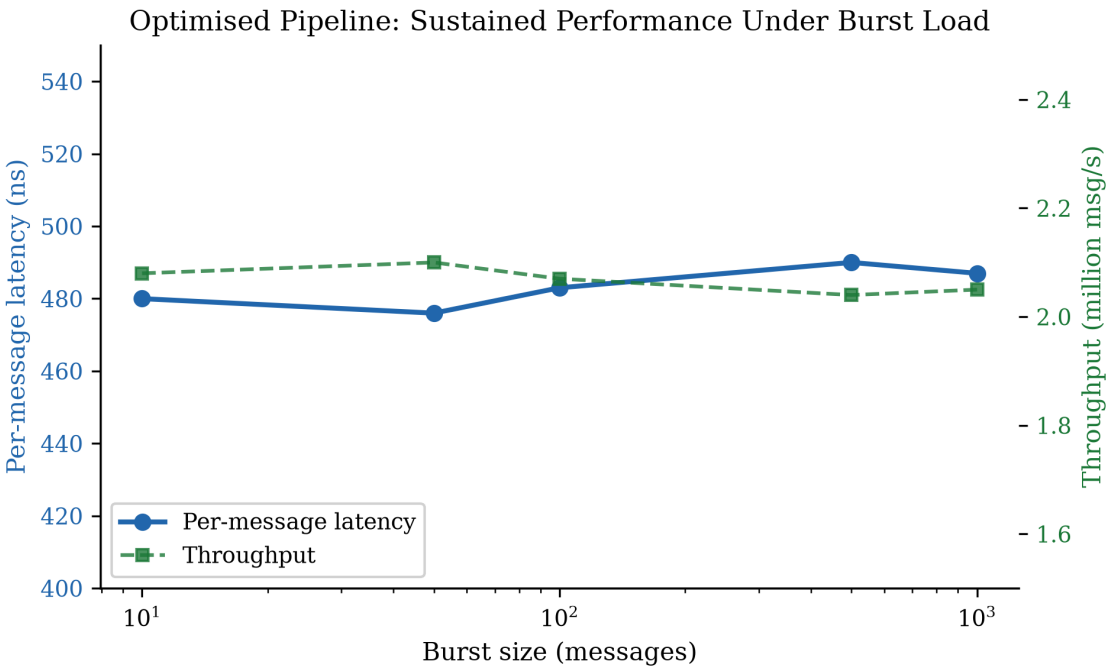


Fig. 5. Optimised pipeline: per-message latency and throughput remain stable across burst sizes of 10 to 1,000 messages (~2.05 M msg/s sustained). Measured data from advanced_examples.cpp, Example C.

This 1.22 ms gap is no longer negligible relative to the ~2 ms network delay. It is the difference between seeing the post-burst order book in near real time and seeing it with a stale, pre-burst snapshot. Put differently, it is the difference between cancelling a stale quote before it is picked off and losing the spread. For a market maker quoting $50,000 BTC at a 2-tick ($0.02) spread, one adverse fill per burst event costs the notional loss on the position; across hundreds of burst events per day, these losses compound into the firm's primary operating cost.

There are limits to this argument, though. It assumes the firm's competitors are running the naive pipeline, which may not be true. If all competitors optimise, the advantage disappears, which is exactly the arms-race dynamic documented by Budish et al. [4]. It also assumes the exchange's snapshot interval does not dominate: if the depth stream updates only once per second, processing each update in 0.5 µs versus 3 µs changes nothing visible. The argument is strongest during periods of genuine burst traffic with sub-100 ms update intervals.

7 Comparative Analysis of Existing Research

Table 10 compares the twelve reviewed studies along four dimensions: what is measured, the market studied, the type of evidence, and whether the findings are directly applicable to our representative firm.

Table 10. Comparison of the twelve reviewed studies and their applicability to a crypto prop firm.

Study	What is measured	Market	Evidence type	Applicability to firm
Albers et al. [1]	Latency gap, execution outcomes	Binance, Bybit (crypto)	Live experiment	Direct: same exchanges
Moallemi & Sağlam [2]	Cost of latency (model)	Equity (theoretical)	Analytical	Framework: cost estimation
Baron et al. [3]	Speed vs. returns	Equity	Empirical	Indirect: different venue
Budish et al. [4]	Arms-race dynamics	Equity	Empirical + model	Indirect: design insight

Study	What is measured	Market	Evidence type	Applicability to firm
Makarov & Schoar [5]	Cross-exchange arbitrage	Crypto	Empirical	Direct: same market
Hautsch et al. [6]	Settlement latency	Crypto	Empirical	Direct: settlement risk
Brogaard et al. [7]	Co-location and liquidity	Equity	Empirical	Indirect: co-location value
Decker & Wattenhofer [8]	BTC propagation delay	Bitcoin network	Empirical	Background only
Gupta et al. [9]	Cloud exchange fairness	Simulated	Prototype	Design insight
Dvořák & Kořenek [10]	FPGA order book	Equity feeds	Laboratory	Design pattern: data structure
Lockwood et al. [11]	FPGA market-data library	Equity feeds	Laboratory	Design pattern: parsing
Høiland-Jørgensen et al. [12]	XDP packet processing	General networking	Throughput test	Indirect: kernel bypass

Three patterns are worth highlighting. First, the strongest direct evidence for cryptocurrency markets comes from a single study: Albers et al. [1] is the only paper that measures execution outcomes on the specific exchanges our firm uses. All other empirical work comes from equity markets [3, 4, 7] or the Bitcoin blockchain [6, 8]. The engineering papers [9–12] measure techniques in laboratory settings with hardware and protocol stacks different from those of cloud-hosted crypto exchanges.

Second, the studies define latency inconsistently. Moallemi and Sağlam [2] model the delay between decision and execution. Albers et al. [1] measure the gap between a snapshot and matching. Lockwood et al. [11] report the delay inside one FPGA appliance. Dvořák and Kořenek [10] measure a single book update. Høiland-Jørgensen et al. [12] report packets per second. These quantities span roughly eight orders of magnitude and cannot be compared directly.

Third, the studies agree on qualitative points but disagree on practical implications. All accept that relative speed matters [1, 3, 4] and that co-location is the first step [1, 3, 7]. The FPGA papers [10, 11] imply that software is too slow and unpredictable for serious HFT. Albers et al. [1] counter that, among co-located cryptocurrency traders, reaction times differ by only a few microseconds while network delay varies by a millisecond, so a race becomes close to a lottery. Our burst-traffic analysis (Section 6) offers a partial resolution: under normal conditions the lottery argument holds, but during burst episodes the software-level difference can compound into a meaningful edge.

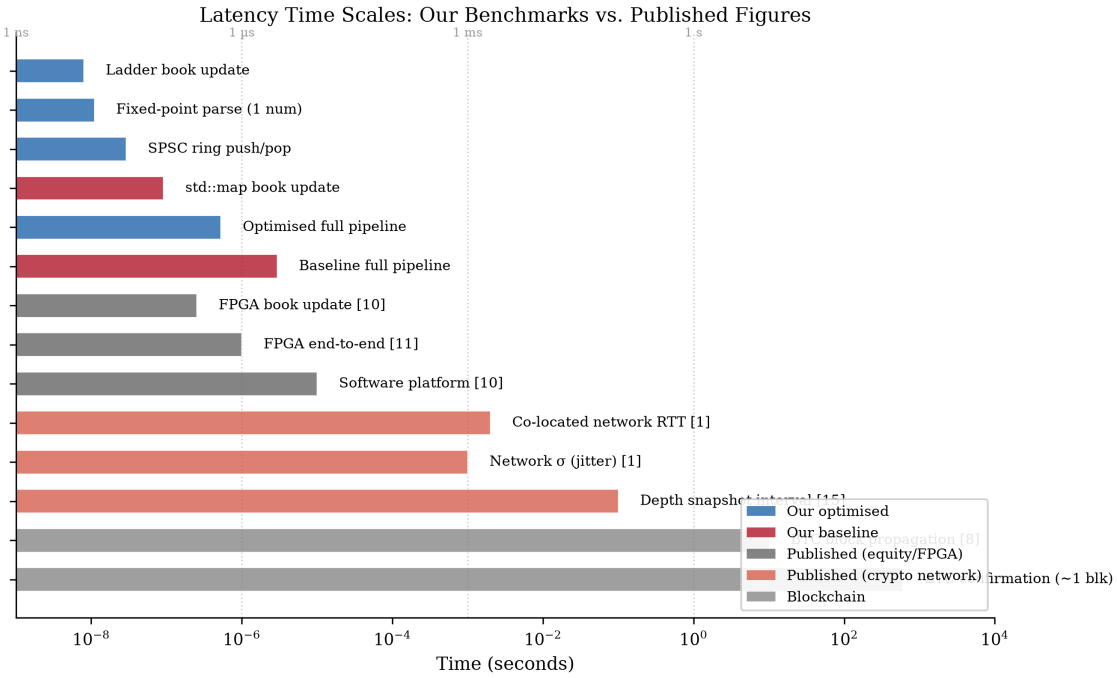


Fig. 6. Time scales reported in the cited sources and measured in this paper. Quantities span roughly ten orders of magnitude and are of different kinds; the axis is for scale, not ranking. Blue = our optimised, red = our baseline, grey = published equity/FPGA, salmon = published crypto network.

8 Limitations and Research Gaps

We should note that this review covers twelve sources and is not exhaustive. For several papers, only the published abstract was available, so our descriptions are limited to what the abstracts report. Proprietary trading firms do not publish their methods; the academic literature almost certainly understates the state of practice.

The benchmarks have their own specific limitations. They run on a desktop computer, not on the AWS instance type the firm would use. The workload is synthetic (messages are generated in memory, not received over a network), so the results exclude kernel-to-userspace copy overhead, TLS decryption, and WebSocket frame reassembly. The batch-timing methodology understates the per-call tail: one slow call in a batch of 100 raises the batch mean by only 1%, so the true p99 is likely worse than reported. Thread hand-off benchmarks use busy-spinning, which gives the best latency but wastes CPU; a production system might use a hybrid spin-then-sleep strategy that trades latency for efficiency.

The representative firm is a composite, not a measurement of a real operation. Real firms face constraints we have not modelled: exchange rate limits, IP bans, API authentication overhead, multi-exchange synchronisation, risk checks before order submission, and the strategy computation itself. The burst-traffic argument in Section 6 is analytical, not measured on a live exchange; the actual message rate during a burst event has not been publicly documented with the precision needed to validate the calculation.

Five specific research gaps deserve attention:

Controlled comparisons. No reviewed study compares latency-reduction techniques on the same cryptocurrency exchange under the same conditions. The engineering results [10–12] come from different hardware, protocols, and years.

Measurement standards. There is no shared methodology for measuring or decomposing end-to-end trading latency on cryptocurrency exchanges. Even Albers et al. [1], the most detailed study, do not publish their full networking configuration.

Software-level evidence in crypto. Language choice, memory management, and concurrency design lack peer-reviewed measurements in a cryptocurrency-trading setting. Our microbenchmarks demonstrate the mechanisms but are not a substitute for live-exchange measurements.

Exchange-side data. The live evidence covers two exchanges, perpetual futures, and periods of days to a week [1]. Spot markets, other exchanges (OKX, Kraken, Coinbase), and longer observation windows remain unstudied.

Cost–benefit analysis. The monetary value of microsecond improvements on cryptocurrency exchanges has not been quantified. Moallemi and Sağlam's [2] cost-of-latency model could be calibrated to Bitcoin data, but no published work has done so.

Realistic next steps include an open benchmark suite that replays recorded exchange data and reports latency by component; controlled A/B comparisons of software configurations from servers in the exchange's cloud region; extension of the Albers et al. [1] experiment to spot markets and additional exchanges; and calibration of a cost-of-latency model [2] to cryptocurrency order-book data.

9 Conclusion

The literature supports a clear ordering of evidence for a proprietary trading firm entering cryptocurrency HFT. Co-location (renting a server in the exchange's cloud region) has the strongest empirical support and should be treated as a prerequisite, not an optimisation. Beyond co-location, the evidence is thinner: hardware acceleration and kernel-bypass networking have strong laboratory results from equity settings but have not been evaluated on cloud-hosted crypto exchanges, and software-level optimisations lack direct evidence in the reviewed literature.

The microbenchmarks we present fill part of that gap. An array-indexed order book is 11× faster than a tree-based one. Zero-copy fixed-point parsing is 3.4× faster than string-based parsing (and 34× faster than a stod-based variant). A lock-free SPSC ring buffer is 2.5× faster than a mutex queue. Across the full pipeline, the optimised path is 5.8× faster than the baseline. These are measured results on one machine with synthetic data; they bound what is achievable in the software layer, not what is achievable end-to-end.

The compound effect becomes operationally significant during burst-traffic episodes, when the firm that drains its message queue fastest recovers an accurate market view before competitors. Our burst-processing benchmark confirmed this directly: the optimised pipeline sustained 2.05 million messages per second and processed a 500-message burst in 0.25 ms versus the baseline's 1.47 ms, a 1.22 ms gap comparable to the network round-trip itself. Whether this advantage translates to profit depends on the competitive landscape: if all firms optimise, the advantage disappears, reproducing the arms-race equilibrium described by Budish et al. [4].

Every technique comes with trade-offs. The price ladder trades memory for speed. The SPSC ring trades generality (one producer, one consumer) for lock-freedom. The zero-copy parser trades flexibility (only the Binance depth format) for speed. Kernel bypass trades OS isolation for control. None is best in every setting, and the most important open task remains direct, controlled measurement on live cryptocurrency exchanges.

References

1. Albers, J., Cucuringu, M., Howison, S. and Shestopaloff, A.Y., 2025. The good, the bad, and latency: exploratory trading on Bybit and Binance. Quantitative Finance, 25(6), pp.919–947. https://doi.org/10.1080/14697688.2025.2515933

2. Moallemi, C.C. and Sağlam, M., 2013. OR Forum: The cost of latency in high-frequency trading. Operations Research, 61(5), pp.1070–1086. https://doi.org/10.1287/opre.2013.1165

3. Baron, M., Brogaard, J., Hagströmer, B. and Kirilenko, A., 2019. Risk and return in high-frequency trading. Journal of Financial and Quantitative Analysis, 54(3), pp.993–1024. https://doi.org/10.1017/S0022109018001096

4. Budish, E., Cramton, P. and Shim, J., 2015. The high-frequency trading arms race: frequent batch auctions as a market design response. The Quarterly Journal of Economics, 130(4), pp.1547–1621. https://doi.org/10.1093/qje/qjv027

5. Makarov, I. and Schoar, A., 2020. Trading and arbitrage in cryptocurrency markets. Journal of Financial Economics, 135(2), pp.293–319. https://doi.org/10.1016/j.jfineco.2019.07.001

6. Hautsch, N., Scheuch, C. and Voigt, S., 2024. Building trust takes time: limits to arbitrage for blockchain-based assets. Review of Finance, 28(4), pp.1345–1381. https://doi.org/10.1093/rof/rfae004

7. Brogaard, J., Hagströmer, B., Nordén, L. and Riordan, R., 2015. Trading fast and slow: colocation and liquidity. The Review of Financial Studies, 28(12), pp.3407–3443. https://doi.org/10.1093/rfs/hhv045

8. Decker, C. and Wattenhofer, R., 2013. Information propagation in the Bitcoin network. IEEE P2P 2013 Proceedings, pp.1–10. https://doi.org/10.1109/P2P.2013.6688704

9. Gupta, E., Goyal, P., Marinos, I., Zhao, C., Mittal, R. and Chandra, R., 2023. DBO: fairness for cloud-hosted financial exchanges. Proceedings of the ACM SIGCOMM 2023 Conference, pp.550–563. https://doi.org/10.1145/3603269.3604871

10. Dvořák, M. and Kořenek, J., 2014. Low latency book handling in FPGA for high frequency trading. 17th International Symposium on Design and Diagnostics of Electronic Circuits and Systems, pp.175–178. https://doi.org/10.1109/DDECS.2014.6868785

11. Lockwood, J.W., Gupte, A., Mehta, N., Blott, M., English, T. and Vissers, K., 2012. A low-latency library in FPGA hardware for high-frequency trading (HFT). 2012 IEEE 20th Annual Symposium on High-Performance Interconnects, pp.9–16. https://doi.org/10.1109/HOTI.2012.15

12. Høiland-Jørgensen, T., Brouer, J.D., Borkmann, D., Fastabend, J., Herbert, T., Ahern, D. and Miller, D., 2018. The eXpress data path: fast programmable packet processing in the operating system kernel. Proceedings of the 14th International Conference on emerging Networking EXperiments and Technologies, pp.54–66. https://doi.org/10.1145/3281411.3281443

13. Dean, J. and Barroso, L.A., 2013. The tail at scale. Communications of the ACM, 56(2), pp.74–80. https://doi.org/10.1145/2408776.2408794

14. Chilimbi, T.M., Hill, M.D. and Larus, J.R., 1999. Cache-conscious structure layout. Proceedings of the ACM SIGPLAN 1999 Conference on Programming Language Design and Implementation, pp.1–12. https://doi.org/10.1145/301618.301633

15. Binance Developer Docs: Spot WebSocket Market Streams, Diff. Depth Stream. https://developers.binance.com/docs/binance-spot-api-docs/web-socket-streams

16. Michael, M.M. and Scott, M.L., 1996. Simple, fast, and practical non-blocking and blocking concurrent queue algorithms. Proceedings of the Fifteenth Annual ACM Symposium on Principles of Distributed Computing, pp.267–275. https://doi.org/10.1145/248052.248106

17. Li, J., Sharma, N.K., Ports, D.R.K. and Gribble, S.D., 2014. Tales of the tail: hardware, OS, and application-level sources of tail latency. Proceedings of the ACM Symposium on Cloud Computing, pp.1–14. https://doi.org/10.1145/2670979.2670988